

Data Analytics Library in C++

An Object-Oriented Design Project

Chiraag K V, Vivek Reddy, Utkarsh Gupta, Russel Gandhi

October 2026

Abstract

This report presents a small data analytics library written in C++20. The library loads tabular data from CSV files into typed columns, computes statistics (mean and median), filters rows using composable condition objects, exports results to CSV and JSON, and renders scatter plots as SVG images. The emphasis of the project is on *object-oriented design*: the system is decomposed into entities with single responsibilities that collaborate through abstract interfaces. The implementation demonstrates inheritance, multiple and virtual inheritance, runtime polymorphism, templates, a custom exception hierarchy, and automatic memory management with smart pointers.

Contents

1	Introduction	2
2	Architecture Summary	2
2.1	Components	2
2.2	Data Flow	4
2.3	Memory Ownership	5
2.4	Project Structure	5
3	Class Definitions	6
3.1	Core: DataSet, Columns, Filter and Conditions	8
3.2	Analytics: Strategy Pattern	10
3.3	I/O and Visualisation	11
3.4	Exceptions	12
4	Usage of OOP Concepts	12
4.1	Abstraction	13
4.2	Encapsulation	13
4.3	Inheritance	13
4.4	Multiple and Virtual Inheritance	13
4.5	Runtime Polymorphism	13
4.6	Templates	14
4.7	Exception Handling	14
4.8	Smart Pointers and Memory Management	15
4.9	Design Patterns	15
5	Results	16
6	Conclusion	16

1 Introduction

A procedural approach to data analysis typically begins with free functions such as `mean()`, `filter()` and `plot()`. While this works for small scripts, data and behaviour remain loosely connected, and supporting a new data type, file format or statistic requires changes in many places.

This project instead starts from the *entities* in the problem domain, gives each one a single responsibility, and connects them through abstract interfaces. New behaviour is added by writing new classes rather than by modifying existing ones.

Objectives

1. Store heterogeneous, typed columns in a single **DataSet**.
2. Compute statistics through interchangeable algorithms.
3. Filter rows with reusable, composable conditions.
4. Import and export data in CSV and JSON formats.
5. Produce a basic scatter-plot visualisation (SVG).
6. Demonstrate core C++ OOP features and manage memory without manual `new/delete`.

2 Architecture Summary

2.1 Components

The library is organised into three layers: a **core** that owns the data, a set of **operations** that read the data but never own it, and an **I/O layer** that moves data between files and memory. Table 1 summarises the components.

Table 1: Main components and their responsibilities

Component	Responsibility	Owns data?	Files
DataSet	Central entity; holds a collection of columns	Yes	DataSet.hpp/.cpp
ColumnBase, Column<T>	A named, typed sequence of values	Yes	ColumnBase.hpp, Column.hpp
Filter	Applies a condition to a named column	No	Filter.hpp/.cpp
Condition & subclasses	A test on a single value, combinable with And/Or/Not	No	Condition.hpp/.cpp
IAnalyzer, StatisticsEngine	Compute statistics (Strategy pattern)	No	Analyzer.hpp/.cpp
IImporter, IExporter	CSV and JSON input/output	No	IO.hpp/.cpp
IVisualizer, ScatterPlot	Render data as an SVG scatter plot	No	Visualizer.hpp/.cpp
DataException & subclasses	Report errors by category	—	Exceptions.hpp

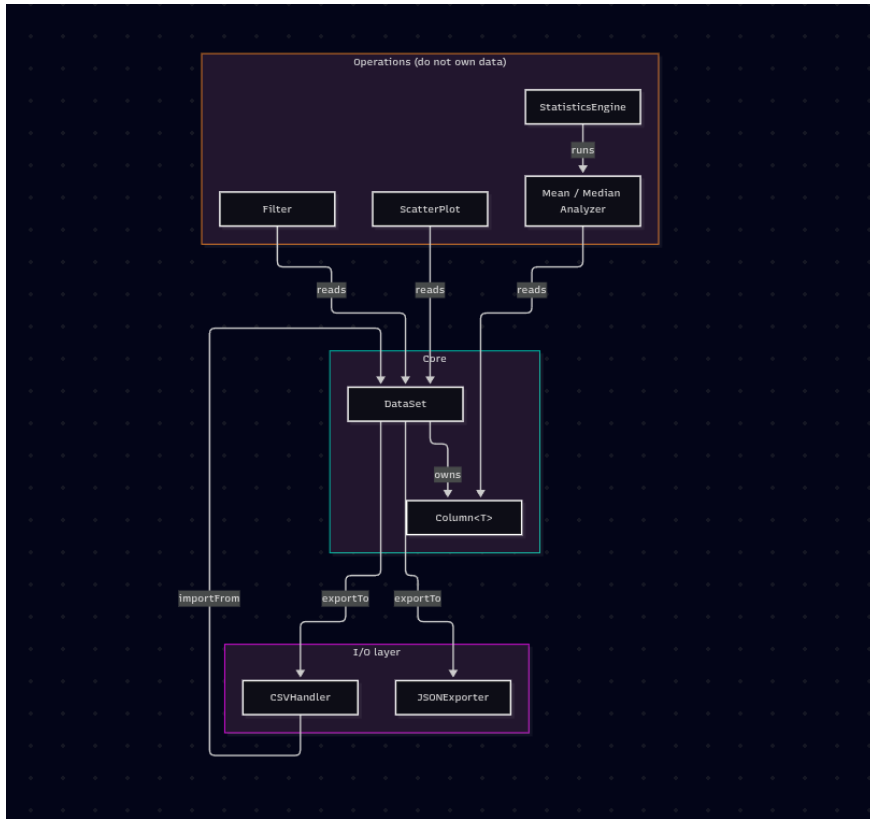


Figure 1: Layered architecture: operations read from the core; the I/O layer imports and exports it.

2.2 Data Flow

Figure 2 shows the typical pipeline executed by the demo program. A CSV file is imported into a `DataSet`; the data set is then analysed, plotted, and filtered, and the filtered result is exported to CSV and JSON.

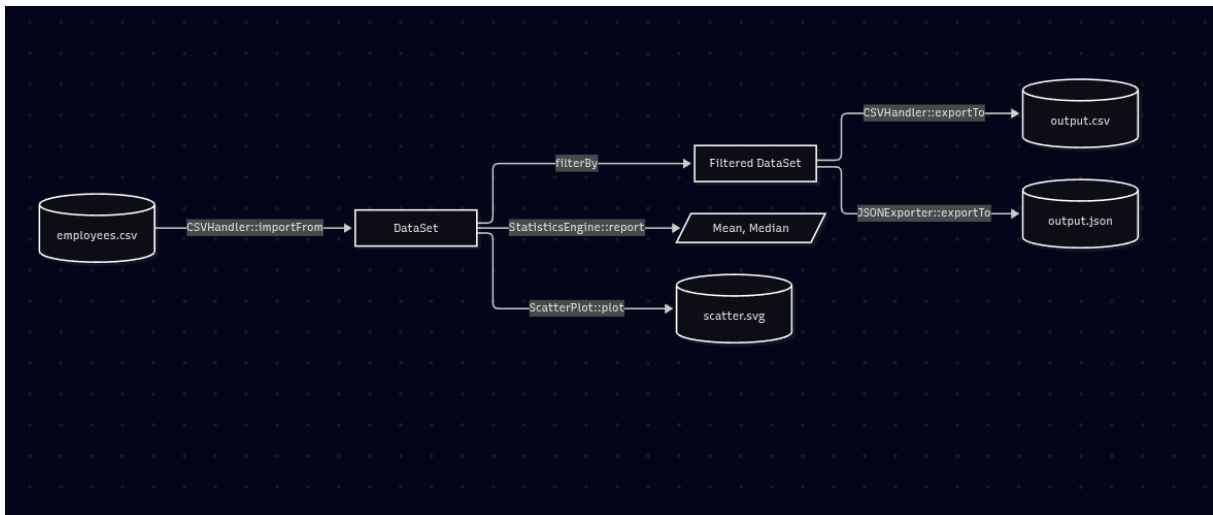


Figure 2: Data flow from CSV input to statistics, SVG plot and exported files.

2.3 Memory Ownership

Ownership is expressed in the type system. A `DataSet` owns its columns and a `StatisticsEngine` owns its analyzers through `std::unique_ptr`. Conditions are the one exception: they are immutable, so a `Filter` and any composite condition hold them through `std::shared_ptr`, which lets filters be copied and reused cheaply. All other access is non-owning, through raw pointers or references (Figure 3).

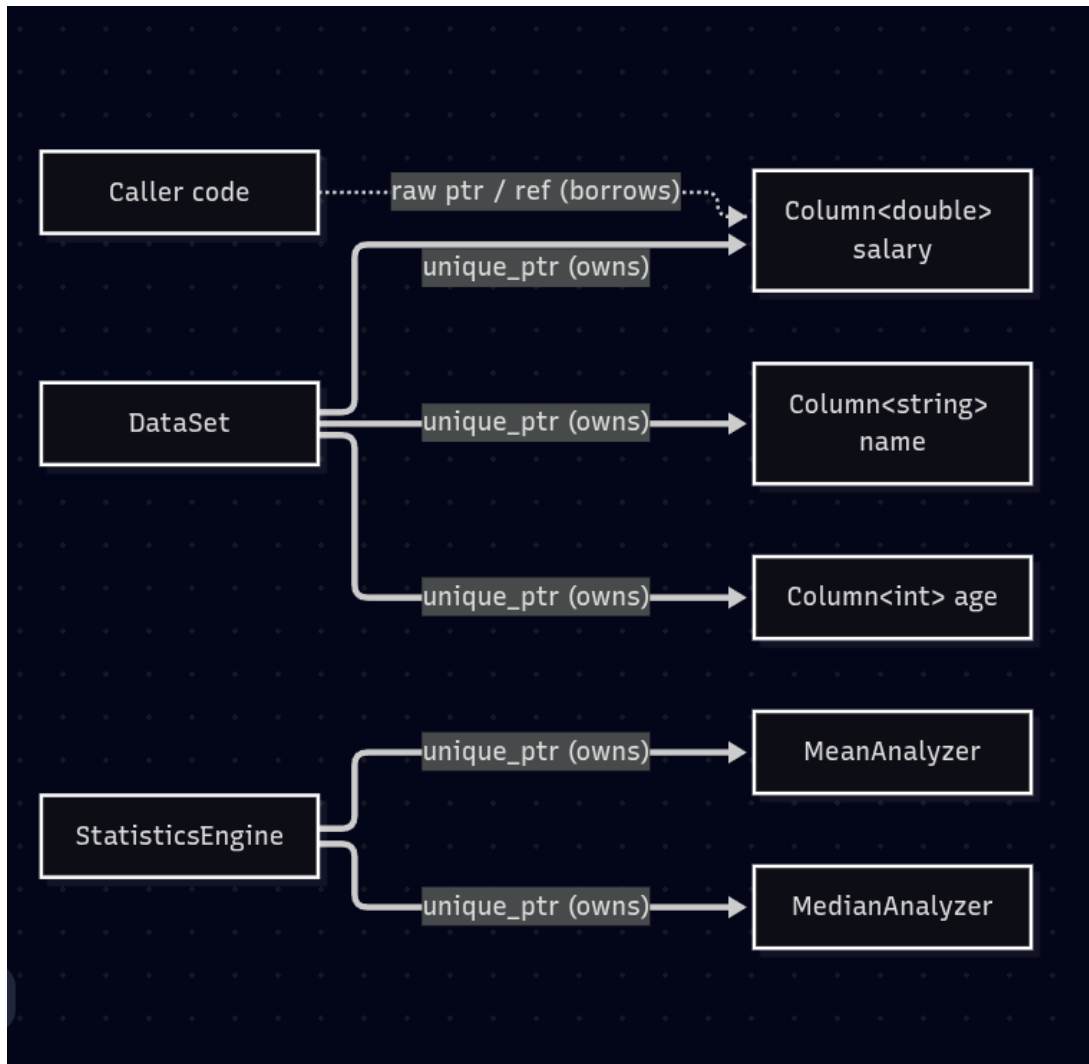


Figure 3: Ownership graph: solid arrows own (`unique_ptr`), thick arrows share ownership (`shared_ptr`), dotted arrows borrow.

2.4 Project Structure

include/	Exceptions.hpp	ColumnBase.hpp	Column.hpp	Condition.hpp
	Filter.hpp	DataSet.hpp	Analyzer.hpp	IO.hpp
	Visualizer.hpp			
src/	DataSet.cpp	Condition.cpp	Filter.cpp	Analyzer.cpp
	IO.cpp	Visualizer.cpp	main.cpp	
data/	employees.csv			
Makefile	(g++ -std=c++20 -Wall -Wextra -pedantic)			

3 Class Definitions

The library's interface is spread over nine header files in `include/`. This section describes what each header contains: its classes, their data members, and the role of each important member function. Each module begins with its UML class diagram. In the diagrams, *italic* methods are pure virtual, `-` marks private members, `#` marks protected members, and `+` marks public members.

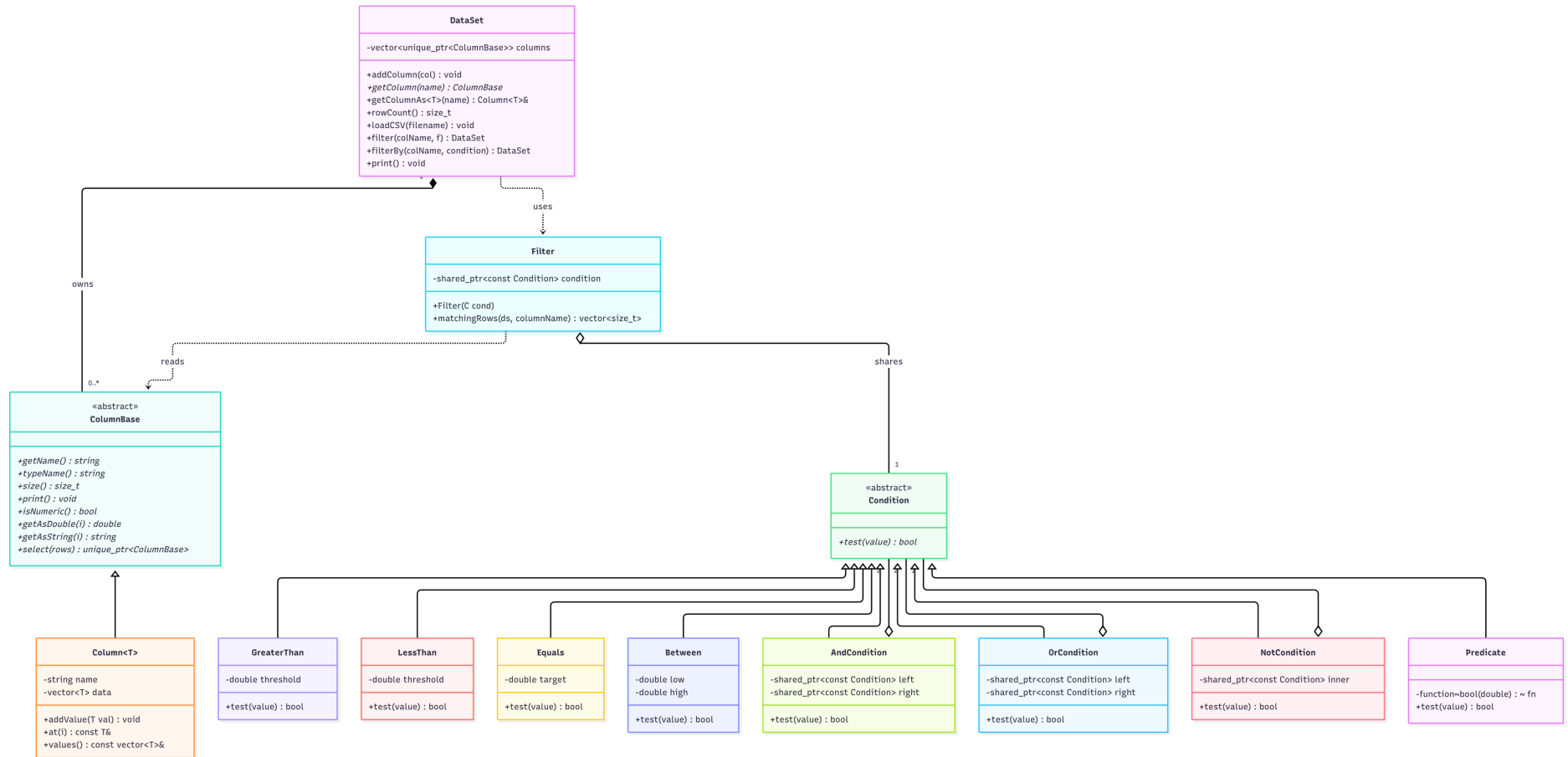


Figure 4: UML class diagram of the core package.

3.1 Core: DataSet, Columns, Filter and Conditions

ColumnBase.hpp

Declares `ColumnBase`, the abstract base class that every column implements. It holds no data; it only defines what the rest of the library may ask of a column, without knowing the column's value type. All of its functions are pure virtual, and it has a virtual destructor so that columns can be safely destroyed through a base-class pointer.

getName(), typeName(), size() Return the column's name, its type as text (for example "int"), and its number of values.

print() Writes the column's name, type and values to the console.

isNumeric(), getAsDouble(i) Let analyzers, filters and the plotter treat any numeric column as a sequence of `double` values.

getAsString(i) Returns one value as text; used by the CSV and JSON exporters.

select(rows) Returns a new column containing only the given row indices; used by filtering.

Column.hpp

Contains the class template `Column<T>`, the only concrete column type, together with the helper trait `TypeName<T>`. Because it is a template, the whole implementation lives in the header and there is no matching `.cpp` file.

TypeName<T> A small struct with specialisations for `int`, `double` and `string` that maps each type to a readable name; any other type is reported as "unknown".

Data members A private `name` and a private `vector<T> data` holding the values.

addValue(val) Appends a value to the column.

at(i), values() Read access: `at` is bounds-checked and throws `std::out_of_range`; `values` returns a read-only reference to the whole vector.

Overrides Implements every `ColumnBase` function. `getAsDouble` uses `if constexpr` so that numeric types are converted with `static_cast` while `Column<string>` throws `TypeMismatchException`. `getAsString` returns strings unchanged and formats numbers with 15 significant digits.

DataSet.hpp

Declares `DataSet`, the central entity of the library. Its only data member is a private `vector<unique_ptr<Column>>` so it owns its columns and can store columns of different types together. It can be moved but not copied, because `unique_ptr` cannot be copied.

addColumn(col) Takes ownership of a column. Rejects null columns, duplicate names, and columns whose length differs from the existing ones.

getColumn(name) Returns a non-owning pointer to a column, or throws `ColumnNotFoundException`. A `const` overload is provided.

getColumnAs<T>(name) Member template that returns the column as a typed `Column<T>&` using `dynamic_cast`; throws `TypeMismatchException` if the column holds a different type.

rowCount(), columnCount(), columnNames(), columnAt(i) Describe the data set's shape and give access to columns by position.

loadCSV(filename) Replaces the contents with data read through `CSVHandler`.

filter(colName, f) Returns a new `DataSet` containing only the rows whose value in `colName` satisfies the filter, for example `ds.filter("age", GreaterThan(30))`. The column is chosen here rather than when the filter is built, so one `Filter` can be reused on any data set or column.

filterBy(colName, condition) Convenience overload for one-off conditions. It uses a C++20 `auto` parameter so any callable can be passed, and wraps it in a `Predicate`.

print() Prints the data set's dimensions and every column.

Filter.hpp

Declares `Filter`, which applies a `Condition` to a column. Its only data member is a `shared_ptr<const Condition>`. The constructor is a template constrained by the `ConditionType` concept, so any `Condition` subclass can be passed by value and is converted implicitly; this is what allows `ds.filter("age", GreaterThan(30))`. Its single operation, `matchingRows(ds, columnName)`, checks that the column exists and is numeric, then returns the indices of the rows for which the condition's `test` is true. `DataSet` is only forward-declared here, which avoids a circular include between the two headers.

Condition.hpp

Declares the abstract class `Condition` and its subclasses, arranged as a Composite (Section 4.9). `Condition` has a virtual destructor and one pure virtual function, `test(value)`, which says whether a single `double` satisfies the condition. The header also defines the concept `ConditionType`, which accepts any type derived from `Condition`.

GreaterThan, LessThan, Equals Compare the value against a stored threshold or target.

Between True when the value lies in the closed range $[low, high]$. The constructor throws `DataException` if `low > high`.

AndCondition, OrCondition Hold two child conditions and combine their results with `&&` or `||`. Evaluation short-circuits.

NotCondition Holds one child condition and negates it; for example `NotCondition(LessThan(30))` expresses “at least 30”.

Predicate Wraps a `function<bool(double)>` for conditions the classes above do not cover. The constructor rejects an empty function.

Composite conditions store their children as `shared_ptr<const Condition>`, built from concept-constrained template constructors, so they nest freely:

```
Filter f = AndCondition(Between(25, 40), NotCondition(Equals(30)));
DataSet a = ds.filter("age", f);
DataSet b = other.filter("experience", f);    // same filter, different data
```

3.2 Analytics: Strategy Pattern

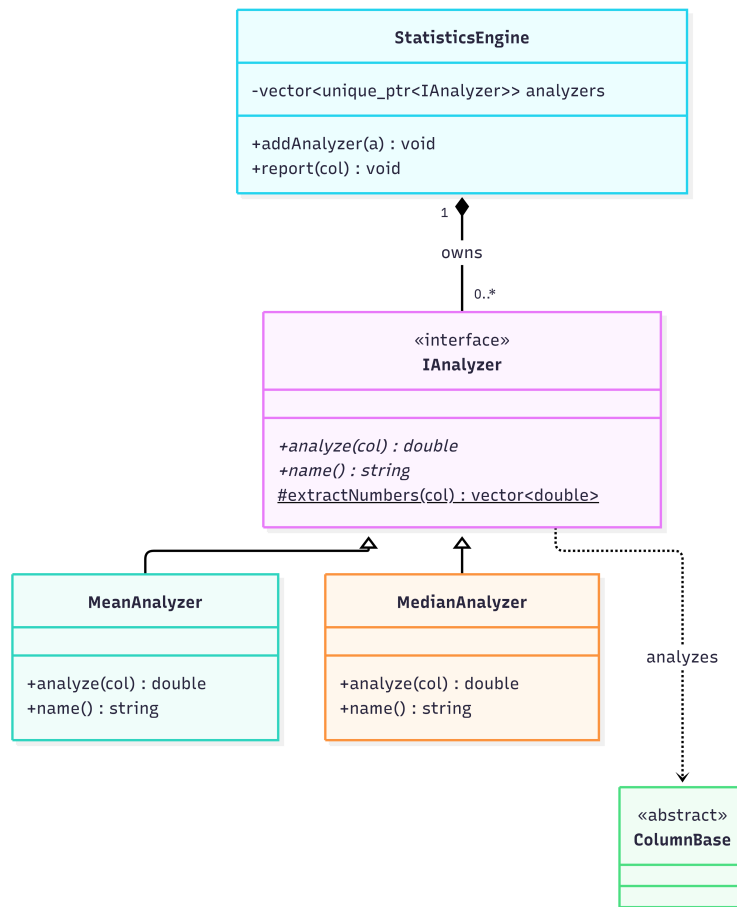


Figure 5: UML class diagram of the analytics package.

Analyzer.hpp

Contains the analytics module, built on the Strategy pattern.

IAnalyzer Abstract strategy interface with two pure virtual functions: `analyze(col)`, which computes a single `double` from a column, and `name()`, which returns the statistic's label. It also provides a protected static helper, `extractNumbers(col)`, which checks that the column is numeric and non-empty and copies its values into a `vector<double>`.

MeanAnalyzer Concrete strategy that returns the arithmetic mean.

MedianAnalyzer Concrete strategy that sorts the values and returns the middle value, or the average of the two middle values when the count is even.

StatisticsEngine Owns a list of analyzers as `unique_ptr<IAnalyzer>`. `addAnalyzer` registers a new strategy, and `report(col)` runs every registered strategy on a column and prints the results.

3.3 I/O and Visualisation

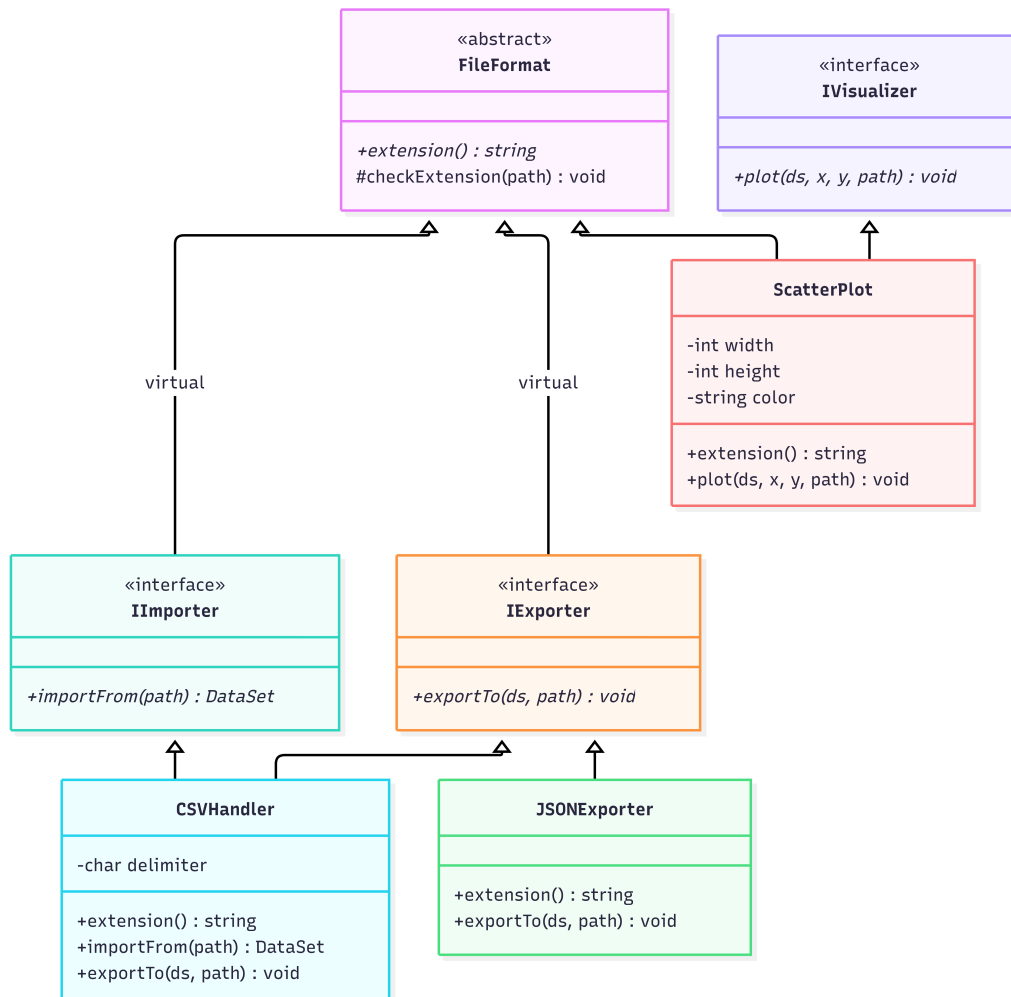


Figure 6: UML class diagram of the I/O and visualisation packages.

IO.hpp

Contains the file input/output classes, arranged as a diamond inheritance hierarchy.

FileFormat Abstract base for anything tied to a file type. It declares the pure virtual `extension()` (for example `".csv"`) and provides the protected helper `checkExtension(path)`, which throws `FileException` when a path has the wrong extension.

IImporter Interface with `importFrom(path)`, which reads a file and returns a new `DataSet`. Inherits `FileFormat` *virtually*.

IExporter Interface with `exportTo(ds, path)`, which writes a `DataSet` to a file. Also inherits `FileFormat` *virtually*.

CSVHandler Inherits both `IImporter` and `IExporter`, so it can read and write CSV. It stores a configurable delimiter (a comma by default). On import it checks that every row has the right number of fields and infers each column's type as `int`, `double` or `string`.

JSONExporter Inherits only `IExporter`. Writes the data set as a JSON array of objects, with numbers unquoted and strings quoted and escaped.

Visualizer.hpp

Contains the visualisation classes.

IVisualizer Interface with the pure virtual `plot(ds, xCol, yCol, path)`, which renders two columns of a data set to a file.

ScatterPlot Inherits both **IVisualizer** and **FileFormat**; its extension is `".svg"`, so the inherited `checkExtension` rejects other output paths. It stores the image width, height and point colour (by default 640×480 pixels in blue). The constructor rejects sizes too small to fit the axis margins. `plot` checks that both columns are numeric, scales the points to pixel coordinates, and writes an SVG file with grid lines, axis ticks and labels, a title, and a hover tooltip on every point.

3.4 Exceptions

Exceptions.hpp

Defines the library's exception hierarchy. Every class is a few lines long, so the file has no matching `.cpp`. Each constructor builds a descriptive message from its arguments.

DataException Base of all library errors; derives from `std::runtime_error` and reuses its constructors. Also thrown directly for general rule violations such as a duplicate column name.

ColumnNotFoundException Takes the missing column's name.

TypeMismatchException Takes the column name and the expected type, for example `"numeric"`.

EmptyDataException Signals that there is no data to work on.

FileException Signals file problems: a missing or unwritable file, a wrong extension, or a malformed CSV row.

4 Usage of OOP Concepts

Table 2 maps each concept to where it is used; the subsections below explain each one with a short excerpt from the code.

Table 2: OOP concepts and where they appear

Concept	Usage in the project
Abstraction	Pure virtual interfaces <code>ColumnBase</code> , <code>Condition</code> , <code>IAnalyzer</code> , <code>FileFormat</code> , <code>IImporter</code> , <code>IExporter</code> , <code>IVisualizer</code>
Encapsulation	Private column data and column list; <code>addColumn</code> enforces invariants
Inheritance	<code>Column<T></code> , condition classes, analyzers, exporters, <code>ScatterPlot</code> , exception classes
Multiple inheritance	<code>CSVHandler : IImporter, IExporter; ScatterPlot : IVisualizer, FileFormat</code>
Virtual inheritance	<code>IImporter</code> and <code>IExporter</code> inherit virtual <code>FileFormat</code>
Runtime polymorphism	Heterogeneous <code>vector<unique_ptr<ColumnBase>></code> ; <code>Condition::test</code> calls from <code>Filter</code> ; analyzer, exporter and visualiser calls through base references
Templates	<code>Column<T></code> , <code>TypeName<T></code> specialisation, <code>if constexpr</code> , <code>getColumnAs<T>()</code> , auto parameter, <code>ConditionType</code> concept
Exception handling	<code>DataException</code> hierarchy; catch by <code>const</code> reference
Smart pointers	<code>unique_ptr</code> and <code>make_unique</code> throughout; <code>shared_ptr</code> for immutable conditions; move-only <code>DataSet</code>
Design patterns	Strategy pattern for analyzers; Composite pattern for conditions

4.1 Abstraction

Interfaces declare *what* an object can do without specifying *how*. A class with at least one pure virtual function (= 0) cannot be instantiated:

```
class IAnalyzer {
public:
    virtual ~IAnalyzer() = default;
    virtual double analyze(const ColumnBase& col) const = 0;
    virtual string name() const = 0;
};
```

The rest of the library (filters, analyzers, exporters, the plotter) depends only on such interfaces, never on concrete column types. Likewise, `Filter` only calls `Condition::test` and never knows which kind of condition it holds.

4.2 Encapsulation

Each column's values are private and modified only through `addValue`; reads go through the bounds-checked `at()`. `DataSet::addColumn` protects the data set's invariants:

```
void DataSet::addColumn(unique_ptr<ColumnBase> col) {
    if (!col) throw DataException("Cannot add a null column");
    for (const auto& c : columns)
        if (c->getName() == col->getName())
            throw DataException("Duplicate column name: " + col->getName());
    if (!columns.empty() && col->size() != rowCount())
        throw DataException("/* row count mismatch */");
    columns.push_back(move(col));
}
```

`IAnalyzer::extractNumbers` is protected: a helper for subclasses, hidden from users.

4.3 Inheritance

- `Column<T>` inherits `ColumnBase`.
- `MeanAnalyzer` and `MedianAnalyzer` inherit `IAnalyzer`.
- `CSVHandler`, `JSONExporter` and `ScatterPlot` inherit their interfaces.
- All custom exceptions inherit `DataException`, which inherits `std::runtime_error`.

4.4 Multiple and Virtual Inheritance

`CSVHandler` can both read and write CSV, so it inherits from both `IImporter` and `IExporter`. Both interfaces derive from `FileFormat`, forming a *diamond*:

```
class FileFormat { public: virtual string extension() const = 0; /* ... */ };
class IImporter : public virtual FileFormat { /* ... */ };
class IExporter : public virtual FileFormat { /* ... */ };
class CSVHandler : public IImporter, public IExporter { /* ... */ };
```

Without the `virtual` keyword, `CSVHandler` would contain two `FileFormat` sub-objects and calls to `extension()` would be ambiguous. Virtual inheritance ensures a single shared base, overridden once. `ScatterPlot` uses ordinary multiple inheritance from `IVisualizer` and `FileFormat` to reuse the extension check for `.svg` files.

4.5 Runtime Polymorphism

A single container holds columns of different concrete types; each call is dispatched through the virtual table to the correct implementation:

```
vector<unique_ptr<ColumnBase>> columns;    // Column<int>, Column<double>, Column<string>

for (const auto& c : columns) c->print(); // calls Column<T>::print for each T
```

The same mechanism runs every analyzer in `StatisticsEngine::report`, every exporter in the demo's export loop, and the plotter through an `IVisualizer`. Every polymorphic base declares a **virtual destructor** so that deleting through a base pointer destroys the derived object correctly, and every override is marked **override** so that signature mistakes are compile-time errors.

4.6 Templates

Class template. `Column<T>` is written once and instantiated for `int`, `double` and `string`. Being a template, it is defined entirely in its header.

Template specialisation. A trait maps each type to a readable name:

```
template <typename T> struct TypeName { static string get() { return "unknown"; } };
template <> struct TypeName<int>      { static string get() { return "int"; } };
template <> struct TypeName<double>   { static string get() { return "double"; } };
```

Compile-time branching. `if constexpr` compiles only the branch valid for `T`, so a string column never attempts a numeric cast:

```
double getAsDouble(size_t i) const override {
    if constexpr (is_arithmetic_v<T>)
        return static_cast<double>(at(i));
    else
        throw TypeMismatchException(name, "numeric");
}
```

Member and abbreviated function templates. `getColumnAs<T>()` returns a typed reference using `dynamic_cast`, and `filterBy` uses a C++20 `auto` parameter to accept any callable:

```
auto& ages = ds.getColumnAs<int>("age");
DataSet even = ds.filterBy("age", [](double a) { return (int)a % 2 == 0; });
```

Concepts. The C++20 concept `ConditionType` restricts the constructor templates of `Filter` and the composite conditions to types derived from `Condition`. Passing anything else is rejected at compile time with a readable error, instead of failing deep inside `make_shared`:

```
template <typename C>
concept ConditionType = derived_from<C, Condition>;

template <ConditionType C>
Filter(C cond) : condition(make_shared<C>(move(cond))) {}
```

4.7 Exception Handling

All library errors derive from `DataException`, allowing callers to catch errors at the level of detail they need:

```
try {
    MeanAnalyzer().analyze(*ds.getColumn("name"));
} catch (const TypeMismatchException& e) { // most specific
```

```

    cout << e.what() << "\n";
} catch (const DataException& e) {           // any library error
    cerr << e.what() << "\n";
} catch (const exception& e) {               // anything else
    cerr << e.what() << "\n";
}

```

Table 3: Exception types and when they are thrown

Exception	Thrown when
ColumnNotFoundException	A column name does not exist
TypeMismatchException	A numeric operation is applied to a string column, or getColumnAs<T> uses the wrong T
EmptyDataException	Analysing an empty column or plotting no rows
FileException	File missing or unwritable, wrong extension, malformed CSV row
DataException	Duplicate column, row-count mismatch, null column or analyzer

4.8 Smart Pointers and Memory Management

The code contains no `new` or `delete`. Objects are created with `make_unique` and ownership transfer is explicit through `move`:

```

auto col = make_unique<Column<double>>("score");
col->addValue(9.5);
ds.addColumn(move(col)); // ownership passes to the DataSet

```

Because `unique_ptr` cannot be copied, `DataSet` is *move-only*; functions such as `filter()` return a new `DataSet` by value, which is moved rather than copied.

Conditions use `shared_ptr<const Condition>` instead. They never change after construction, so sharing them is safe, and it makes `Filter` and the composite conditions cheap to copy: a copy shares the same condition tree rather than duplicating it.

4.9 Design Patterns

Strategy

Each analyzer is an interchangeable strategy. Adding a new statistic, such as standard deviation, requires only one new class overriding `analyze()`; no existing class changes, in line with the Open/Closed Principle.

```

StatisticsEngine engine;
engine.addAnalyzer(make_unique<MeanAnalyzer>());
engine.addAnalyzer(make_unique<MedianAnalyzer>());
engine.report(*ds.getColumn("salary"));

```

Composite

Conditions follow the Composite pattern. Simple conditions (`GreaterThan`, `Between`, ...) are leaves; `AndCondition`, `OrCondition` and `NotCondition` are composites that hold other conditions and are themselves conditions. Clients treat both the same way, by calling `test()`, so arbitrarily nested expressions can be built without changing `Filter`. A new kind of condition only needs a new subclass that overrides `test()`.

```

// 25 <= age <= 40, excluding 30, or anyone over 55
Filter f = OrCondition(AndCondition(Between(25, 40), NotCondition(Equals(30))),
    GreaterThan(55));

```

5 Results

The demo program (`src/main.cpp`) exercises every feature on a sample of ten employees. CSV type inference correctly produced `string` columns for *name* and *department*, `int` columns for *age* and *experience*, and a `double` column for *salary*.

Table 4: Statistics computed by `StatisticsEngine`

Column	Mean	Median
salary	78700.1	75500.2
age	33.5	32

Filtering the *age* column with `GreaterThan(30)` returned 6 of the 10 rows, which were exported to `output.csv` and `output.json`. Every deliberate error case (missing column, numeric operation on a string column, wrong typed access, missing file, wrong plot extension, duplicate column) raised the expected exception. The project compiles without warnings under `-Wall -Wextra -pedantic`.

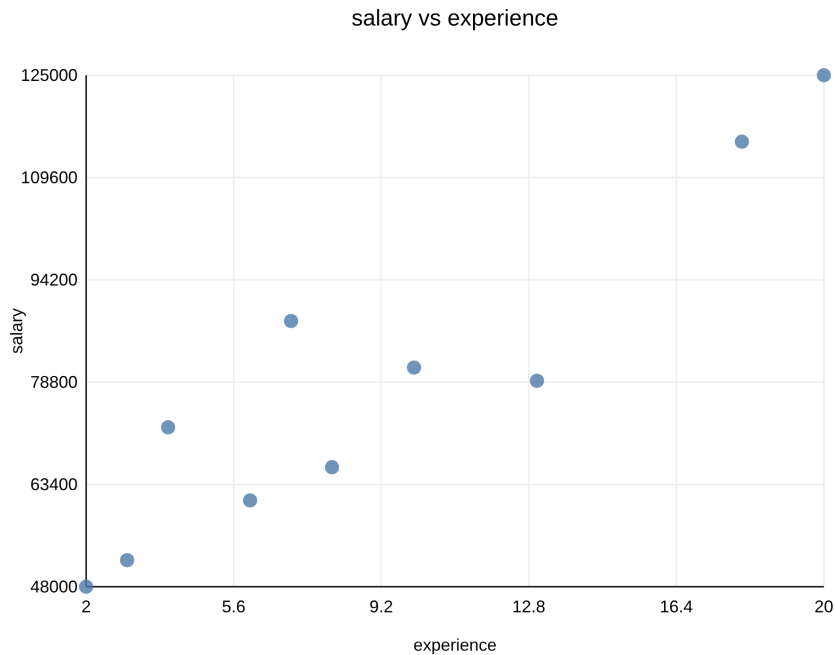


Figure 7: SVG scatter plot of salary against experience, produced by `ScatterPlot`.

6 Conclusion

Designing around entities and interfaces rather than free functions produced a library that is easy to extend and safe to use. New statistics, file formats and chart types can each be added as a single new class without modifying existing code, and filter conditions can be combined freely through the Composite pattern. Smart pointers eliminate manual memory management, and the exception hierarchy makes failures explicit and easy to handle.

Limitations and future work. Filters currently support only numeric columns, the CSV parser does not handle quoted fields, and JSON is export-only. Additional analyzers (stan-

dard deviation, correlation), chart types (line, bar) and conditions can be added by subclassing `IAnalyzer`, `IVisualizer` and `Condition`.

References

- [1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.
- [2] cppreference.com, *C++ Reference*. <https://en.cppreference.com>